CandlestickPlotter.cpp

```cpp
#include "CandlestickPlotter.h"
#include <iostream>
#include <iomanip>
#include <cmath>
#include <tuple>
#include <string>
#include <vector>
#include <sstream>
#include <algorithm>

//code part without assisstance START
// ANSI escape codes for colors that i used for the candlesticks
#define RED "\033[31m"
#define GREEN "\033[32m"
#define RESET "\033[0m"

//normalize a value for ploting to a scale
int normalize(double value, double min, double max, int scale) {
    double normalizedValue = (value - min) / (max - min) * scale;
    int result = std::round(normalizedValue); // Round to avoid off-by-one errors
    return result;
}

void CandlestickPlotter::plotCandlesticks(const
std::vector<std::vector<std::string>>& ohlcData) {
    const int plotHeight = 50;        //  height of the plot
    const int candlestickSpacing = 8; // spacing for compactnesss
    const double bufferFactor = 0.1; // 10% buffer above max and below min

    if (ohlcData.empty()) {
        std::cerr << "No data to plot!\n";
        return;
    }

    // displayingg the OHLC table for referencce
    std::cout << "\n================= OHLC Table for plotting =================\n";
    std::cout << std::setw(12) << "Date"
              << std::setw(12) << "Open"
              << std::setw(12) << "High"
              << std::setw(12) << "Low"
              << std::setw(12) << "Close" << "\n";

    //print each row of OHLC data
    for (const auto& row : ohlcData) {
        std::cout << std::setw(12) << row[0]
                  << std::setw(12) << std::fixed << std::setprecision(3) << row[1]
                  << std::setw(12) << row[2]
                  << std::setw(12) << row[3]
                  << std::setw(12) << row[4] << "\n";
    }
    std::cout << "============================================\n";

```

```
52      double globalHigh = -std::numeric_limits<double>::infinity();
53      double globalLow = std::numeric_limits<double>::infinity();
54  //code part without assisstance END
55
56
57  //code with a little teachers assisstance START
58      // Calculate global min and max
59      for (const auto& row : ohlcData) {
60          double high = std::stod(row[2]);
61          double low = std::stod(row[3]);
62          globalHigh = std::max(globalHigh, high);
63          globalLow = std::min(globalLow, low);
64      }
65
66      // add buffer to ensure the plot looks nicee
67      double range = globalHigh - globalLow;
68      if (range == 0) range = 1;
69      double buffer = range * bufferFactor;
70      globalHigh += buffer;
71      globalLow -= buffer;
72
73      // normalize candlestick positions for plotting
74      std::vector<std::tuple<int, int, int, int>> normalizedCandlesticks;
75      for (const auto& row : ohlcData) {
76          double open = std::stod(row[1]);
77          double high = std::stod(row[2]);
78          double low = std::stod(row[3]);
79          double close = std::stod(row[4]);
80
81          int highPos = normalize(high, globalLow, globalHigh, plotHeight);
82          int lowPos = normalize(low, globalLow, globalHigh, plotHeight);
83          int openPos = normalize(open, globalLow, globalHigh, plotHeight);
84          int closePos = normalize(close, globalLow, globalHigh, plotHeight);
85
86          normalizedCandlesticks.emplace_back(highPos, lowPos, openPos, closePos);
87      }
88
89      // generating the candlestick plotting thrugh a loop
90      std::cout << "\n=============== Plotting the Candlestick==============\n";
91      for (int i = plotHeight; i >= 0; --i) {
92          // label the yaxis with the corespondingg values
93          double labelValue = globalLow + (globalHigh - globalLow) * i / plotHeight;
94          std::cout << std::setw(6) << std::fixed << std::setprecision(1) <<
    labelValue << " |";
95
96          for (size_t j = 0; j < ohlcData.size(); ++j) {
97              auto [highPos, lowPos, openPos, closePos] = normalizedCandlesticks[j];
98              bool isUpward = closePos > openPos; // True if green candlestick
99
100             std::string color = isUpward ? GREEN : RED;
101
102             // spacinfg out the candlesticks and aligning  them above their
    corresponding date
103             if (j > 0) {
```

```
104                 std::cout << std::string(candlestickSpacing - 4, ' '); // Adjust
    spacing for alignment
105             }
106 
107             // rendering body (Open to Close) of the candlestick
108             if (i <= std::max(openPos, closePos) && i >= std::min(openPos,
    closePos)) {
109                 std::cout << color << " ▇▇▇  " << RESET;
110             }
111             //rrendering the wick (High to Low) out of body area
112             else if (i <= highPos && i > lowPos) {
113                 std::cout << color << "  │   " << RESET;
114             }
115             //rendering  the empty space
116             else {
117                 std::cout << "      ";
118             }
119         }
120         std::cout << "\n";
121     }
122 // code with a little teachers assisstance STOP
123 
124 
125     //draw X-axis
126     std::cout << "       +";
127     for (size_t j = 0; j < ohlcData.size(); ++j) {
128         std::cout << std::string(candlestickSpacing, '-'); // proper alignment for
    X-axis
129     }
130     std::cout << "\n";
131 
132     // draw X-axis labels with reduce gap b/w them
133     std::cout << "       ";
134     for (size_t j = 0; j < ohlcData.size(); ++j) {
135         std::string shortDate = ohlcData[j][0].substr(2, 8); // formatingg to "YY-
    MM-DD"
136         std::cout << std::string(candlestickSpacing - 6, ' ') << shortDate; //
    alignn labels
137     }
138     std::cout << "\n";
139 }
140 
```

CandlestickPlotter.h

```h
#ifndef CANDLESTICK_PLOTTER_H
#define CANDLESTICK_PLOTTER_H

#include <vector>
#include <string>

// This class is responsible for plotting candlestick charts
class CandlestickPlotter {
public:
    // Constructor of plotting the candlestick chart
    void plotCandlesticks(const std::vector<std::vector<std::string>>& ohlcData);
};

#endif

// this part of code is written without assisstance
```

CSVReader.cpp

```cpp
#include "CSVReader.h"
#include <fstream>
#include <sstream>
#include <iostream>

// constructor
CSVReader::CSVReader() {}

// tokenize a CSV line based on a separator
std::vector<std::string> CSVReader::tokenise(const std::string& csvLine, char separator) {
    std::vector<std::string> tokens;
    std::stringstream ss(csvLine);
    std::string token;

    while (std::getline(ss, token, separator)) {
        tokens.push_back(token);
    }

    return tokens;
}

//readd CSV file andd returns it as a 2D vector
std::vector<std::vector<std::string>> CSVReader::readCSV(const std::string& inputFile) {
    std::ifstream csvFile(inputFile);
    std::vector<std::vector<std::string>> data;
    std::string line;

    if (!csvFile.is_open()) {
        std::cerr  << "Could not open the file: " << inputFile << std::endl;
        return data;
    }

    while (std::getline(csvFile, line)) {
        data.push_back(tokenise(line, ',')); //adjust separator if needed
    }

    csvFile.close();
    return data;
}

//extract country codes from header of CSV
std::vector<std::string> CSVReader::getCountries(const std::vector<std::string>& header) {
    std::vector<std::string> countries;
    for (const auto& col : header) {
        if (col.find("_temperature") != std::string::npos) {
            countries.push_back(col.substr(0, col.find("_temperature")));
        }
    }
    return countries;
```

```
}

//this part of code was written without assisstance
```

CSVReader.h

```h
#ifndef CSVREADER_H
#define CSVREADER_H

#include <vector>
#include <string>

//class to handle reading and thee processing of CSV data
class CSVReader {
public:
    CSVReader();
    std::vector<std::vector<std::string>> readCSV(const std::string& inputFile);
    std::vector<std::string> getCountries(const std::vector<std::string>& header);
    std::vector<std::vector<std::string>> calculateOHLC(const std::vector<std::vector<std::string>>& data, const std::string& countryCode);

private:
//thiss splits a CSV line into tokens based on a separator
    std::vector<std::string> tokenise(const std::string& csvLine, char separator);
};

#endif

//code part written without assistancee
```

main.cpp

```cpp
#include "CSVReader.h"
#include "OHLCComputer.h"
#include "CandlestickPlotter.h"
#include "PredictionSystem.h"
#include <iostream>
#include <iomanip>
#include <limits>
#include <string>


// code part without assisstance START
// displayingg the main menu in the start
void displayMainMenu() {
    std::cout << "\n========== WEATHER ANALYSIS DATA IMPLEMENTATIONS ==========\n";
    std::cout << "1. View the filtered OHLC table and Candlestick plot \n";
    std::cout << "2. Prediction (Task 4)\n";
    std::cout << "0. Exit\n";
    std::cout << "=======================================\n";
}

//displaying the country sellection menu
void displayMenu(const std::vector<std::string>& countries) {
    std::cout << "\n========== Countries To Choose From==========\n";
    for (size_t i = 0; i < countries.size(); ++i) {
        std::cout << i + 1 << ". " << countries[i] << "\n";
    }
    std::cout << "0. Back\n";
    std::cout << "=========================================\n";
}

//here display the resolution menu
void displayResolutionMenu() {
    std::cout << "\n========== Select the Time Resolution ==========\n";
    std::cout << "1. Daily\n";
    std::cout << "2. Monthly\n";
    std::cout << "3. Yearly\n";
    std::cout << "0. Go Back\n";
    std::cout << "=========================================\n";
}

//Clear input buffer to handle invalid user input
void clearInput() {
    std::cin.clear();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

// Get valid input from the userr
int getValidInput(int min, int max) {
    int choice;
    while (true) {
        std::cin >> choice;
```

```
52          if (std::cin.fail() || choice < min || choice > max) {
53              std::cout << "Invalid input. Please enter a number between " << min << "
    and " << max << ": ";
54              clearInput();
55          } else {
56              break;
57          }
58      }
59      return choice;
60  }
61
62  //CODE PART WITHOUT ASSISTANCE END
63
64
65
66  //this part of code below needed some asssistance, the main function as creating the
    main menu was a bit tricky
67  //the main functionn
68  int main() {
69
70        // Initialize required classes
71      CSVReader reader;
72      OHLCComputer ohlcComputer;
73      CandlestickPlotter plotter;
74      PredictionSystem predictor;
75      std::string inputFile = "weather_data_EU_1980-2019_temp_only.csv";
76
77      //reading the CSV data
78      auto data = reader.readCSV(inputFile);
79      if (data.empty()) {
80          std::cerr << "Error: Unable to read the file: " << inputFile << std::endl;
81          return 1;
82      }
83
84      //extracting the countries from the header
85      const auto& header = data[0];
86      auto countries = reader.getCountries(header);
87
88      while (true) {
89          displayMainMenu();
90          std::cout << "Select an option: ";
91          int mainChoice = getValidInput(0, 2);
92
93          if (mainChoice == 0) {
94              std::cout << "Exiting program.. Goodbye!\n";
95              break;
96          }
97          //displaying main menu and handling user choice
98          displayMenu(countries);
99          std::cout << "Select a country by number (or 0 to go back): ";
100         int countryChoice = getValidInput(0, countries.size());
101         if (countryChoice == 0) {
102             continue;
103         }
```

```
104
105         std::string selectedCountry = countries[countryChoice - 1];
106         std::cout << "\nYou selected: " << selectedCountry << "\n";
107
108         if (mainChoice == 1) {
109              // viewing OHLC data  and candlestick plottingg
110             while (true) {
111                 displayResolutionMenu();
112                 std::cout << "Select a resolution by number (or 0 to go back): ";
113                 int resolutionChoice = getValidInput(0, 3);
114                 if (resolutionChoice == 0) {
115                     break;
116                 }
117
118                 std::string resolution;
119                 if (resolutionChoice == 1) resolution = "daily";
120                 else if (resolutionChoice == 2) resolution = "monthly";
121                 else resolution = "yearly";
122
123                 std::string startDate, endDate;
124                 std::cout << "Enter the start date (YYYY-MM-DD): ";
125                 std::cin >> startDate;
126                 std::cout << "Enter the end date (YYYY-MM-DD): ";
127                 std::cin >> endDate;
128
129                 auto ohlcData = ohlcComputer.calculateOHLC(data, selectedCountry,
    resolution, startDate, endDate);
130                 if (ohlcData.empty()) {
131                     std::cout << "No OHLC data available for " << selectedCountry <<
    " within the specified date range.\n";
132                 } else {
133                     std::cout << "\nOHLC Data (" << resolution << ") for " <<
    selectedCountry << ":\n";
134                     plotter.plotCandlesticks(ohlcData);
135                 }
136             }
137         } else if (mainChoice == 2) {
138             //predicting task
139             std::cout << "\n========== Predict Weather ==========\n";
140
141             // predict for yearly resolution starting from 2020
142             auto ohlcData = ohlcComputer.calculateOHLC(data, selectedCountry,
    "yearly", "1980-01-01", "2019-12-31");
143             if (ohlcData.empty()) {
144                 std::cout << "No data available for " << selectedCountry << "\n";
145                 continue;
146             }
147
148             std::cout << "Enter the number of future years to predict: ";
149             int yearsToPredict = getValidInput(1, 10);
150
151             int movingAverageWindow = static_cast<int>(ohlcData.size() / 5);
152             if (movingAverageWindow == 0) movingAverageWindow = 1;
153
```

```
154             auto predictionData = predictor.predictTemperature(ohlcData,
    yearsToPredict, movingAverageWindow);
155 
156             std::cout << "\nPredicted OHLC Data for " << selectedCountry << ":\n";
157 
158             // Plot predicted data
159             plotter.plotCandlesticks(predictionData);
160         }
161     }
162 
163     return 0;
164 }
165 
```

OHLCComputer.cpp

```cpp
#include "OHLCComputer.h"
#include <map>
#include <algorithm>
#include <numeric>
#include <iomanip>
#include <iostream>
#include <sstream>

// function that help s to extract date from UTC timestamp
std::string extractDate(const std::string& utcTimestamp) {
    return utcTimestamp.substr(0, 10); // Extract "YYYY-MM-DD"
}

//  function that helps to create group keys based on resolution
std::string createGroupKey(const std::string& date, const std::string& resolution) {
    if (resolution == "daily") {
        return date; // YYYY-MM-DD
    } else if (resolution == "monthly") {
        return date.substr(0, 7); // YYYY-MM
    } else if (resolution == "yearly") {
        return date.substr(0, 4); // YYYY
    }
    return date; // default to daily if the resolution is invalid
}

//helper function to Check if  date is within the specified start and end date range
bool isWithinRange(const std::string& date, const std::string& startDate, const
std::string& endDate) {
    return date >= startDate && date <= endDate;
}

// calculates OHLC (Open, High, Low, Close) data for the given country and resolution
std::vector<std::vector<std::string>> OHLCComputer::calculateOHLC(
    const std::vector<std::vector<std::string>>& data,
    const std::string& countryCode,
    const std::string& resolution,
    const std::string& startDate,
    const std::string& endDate
) {
    std::map<std::string, std::vector<double>> groupedTemperatures;

// this identifiess the column index for the target country temperature data
    size_t countryColIndex = 0;
    const auto& header = data[0];
    std::string targetColumn = countryCode + "_temperature";
    for (size_t i = 0; i < header.size(); ++i) {
        if (header[i] == targetColumn) {
            countryColIndex = i;
            break;
        }
    }

```

```
52      // Group tempp data by resolution (daily, monthly, yearly)
53      for (size_t i = 1; i < data.size(); ++i) {
54          const std::string& utcTimestamp = data[i][0];
55          std::string date = extractDate(utcTimestamp);
56
57          double temp = std::stod(data[i][countryColIndex]);
58          std::string groupKey = createGroupKey(date, resolution);
59
60          if (date < startDate) {
61              groupedTemperatures[groupKey].push_back(temp); // Store temps before
    startDate for prevClose
62          } else if (isWithinRange(date, startDate, endDate)) {
63              groupedTemperatures[groupKey].push_back(temp);
64          }
65      }
66
67      // find the last Close value before the startDate
68      double prevClose = 0.0;
69      for (const auto& [key, temps] : groupedTemperatures) {
70          if (key < startDate) {
71              prevClose = std::accumulate(temps.begin(), temps.end(), 0.0) /
    temps.size();
72          }
73      }
74
75      std::vector<std::vector<std::string>> ohlcData;
76
77      //generating OHLC data for the specified range
78      for (const auto& [key, temps] : groupedTemperatures) {
79          if (key < startDate || key > endDate) continue;
80
81          double high = *std::max_element(temps.begin(), temps.end());
82          double low = *std::min_element(temps.begin(), temps.end());
83          double close = std::accumulate(temps.begin(), temps.end(), 0.0) /
    temps.size();
84          double open = (prevClose == 0.0) ? close : prevClose; // Use prevClose or
    default to close
85
86          ohlcData.push_back({
87              key,
88              std::to_string(open),
89              std::to_string(high),
90              std::to_string(low),
91              std::to_string(close)
92          });
93
94          prevClose = close; // update previous close for the next group
95      }
96
97      return ohlcData;
98  }
99
```

OHLCComputer.h

```h
#ifndef OHLC_COMPUTER_H
#define OHLC_COMPUTER_H

#include <vector>
#include <string>

//calculating the  OHLC (Open, High, Low, Close) data from temperature records
class OHLCComputer {
public:

    // Calculates OHLC data for a country using the given data
    // Takes the country code, resolution (daily, monthly, yearly), start date and end date
    // Returns a 2D vector with OHLC data (date, open, high, low, close)
    std::vector<std::vector<std::string>> calculateOHLC(
        const std::vector<std::vector<std::string>>& data,
        const std::string& countryCode,
        const std::string& resolution,
        const std::string& startDate,
        const std::string& endDate
    );
};

#endif

// this part of code was written without assistance

```

PredictionSystem.cpp

```cpp

#include "PredictionSystem.h"
#include <vector>
#include <string>
#include <sstream>
#include <iomanip>
#include <iostream>
#include <numeric>
#include <algorithm>

// part of code done a a little assistancee START
// function that helpss to convert a year string to an integer
int yearToInt(const std::string& year) {
    return std::stoi(year);
}

//function that helps to to create a year string from an integer
std::string intToYear(int year) {
    return std::to_string(year);
}

//predicts future temperature data based on previouss OHLC data
std::vector<std::vector<std::string>> PredictionSystem::predictTemperature(
    const std::vector<std::vector<std::string>>& historicalOHLC,
    int yearsToPredict,
    int movingAverageWindow
) {
    std::vector<std::vector<std::string>> predictedData;

    // Initialize with the last year of historical data
    auto lastRow = historicalOHLC.back();
    double lastClose = std::stod(lastRow[4]);

    // initialize sums for calculating averages
    double sumOpen = 0.0, sumHigh = 0.0, sumLow = 0.0, sumClose = 0.0;
    int count = historicalOHLC.size();

    for (const auto& row : historicalOHLC) {
        sumOpen += std::stod(row[1]);
        sumHigh += std::stod(row[2]);
        sumLow += std::stod(row[3]);
        sumClose += std::stod(row[4]);
    }
    // part of code done a a little assistancee END


// calculated the mean avg w/out assistance
    //predicting for future years
    int startYear = std::stoi(lastRow[0]);
    for (int i = 1; i <= yearsToPredict; ++i) {
        double predictedOpen = lastClose;
```

```
        double predictedHigh = sumHigh / count;
        double predictedLow = sumLow / count;
        double predictedClose = sumClose / count;

        //adding predicted data for the current year
        predictedData.push_back({
            std::to_string(startYear + i),
            std::to_string(predictedOpen),
            std::to_string(predictedHigh),
            std::to_string(predictedLow),
            std::to_string(predictedClose)
        });

        //update cumulative sums and count
        sumOpen += predictedOpen;
        sumHigh += predictedHigh;
        sumLow += predictedLow;
        sumClose += predictedClose;
        count++;

        //Update lastClose for continuity
        lastClose = predictedClose;
    }

    return predictedData;
}

```

PredictionSystem.h

```h
#ifndef PREDICTION_SYSTEM_H
#define PREDICTION_SYSTEM_H

#include <vector>
#include <string>

//code part done with some teacher assistanccee

//class for predicting future temperature data
class PredictionSystem {
public:

    // predicts future temperatures based on historical OHLC data
    // takess the data, number of years to predict and moving average window size
    // Returns a 2D vector with predicted OHLC dataa
    std::vector<std::vector<std::string>> predictTemperature(
        const std::vector<std::vector<std::string>>& ohlcData,
        int yearsToPredict,
        int movingAverageWindow
    );
};

//function tto convert a year string to an integer
int yearToInt(const std::string& year);

#endif

```